LAB 5 – Assignment

Name: Krish Singh

Roll: AID25072

Class: AID-G sem 1

Experiment 1: Object Creation Without Constructor

Program:

```
class Student{
   int id;
   String name;

   void display(){
      System.out.println(id+" "+name);
   }
}

public class Test1{
   public static void main(String[] test1){
      Student s1 = new Student();
      s1.display();
   }
}
```

Explanation:

Logic : so what we see here is that we have a class students which contains instance variables id of int type and name
of string type, we have a method display() that prints id and name. Then we have a public class Test1, inside of which
an object s1 of class Student is created. Here An object of the Student class is created using implicit default constructor provided by java compiler.
When the object is created, that line is executed, memory is allocated for the object and the instance variables are automatically initialized with default values because no constructor is defined in the class.
Output: We get '0 null' as output because we didn't initialize any value to the variables. So the object is created with
default values.

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java
0 null

Process finished with exit code 0
```

Experiment 2: Default Constructor

Program:

```
class Student2{
   int id;
   String name;

   Student2(){
      id=101;
      name="Asha";
   }

   void display(){
      System.out.println(id+" "+name);
   }
}
public class Test2{
   public static void main(String[] test2){
      Student2 s1 = new Student2();
      s1.display();
   }
}
```

Explanation:

LOGIC: here we have a class Student2 with instance variables id and name of type integer and String respectively.
we have a class constructor with same name as the class Student2 which initializes values for id and name as 101 and Aasha respectively.
we have a method defined as display() that prints the values stored in id and name.
Then we have class Test2 with the main method inside which an object is created of Student2 class with object refrence s1.

Working: When the object is created it triggers default constructor Student2() which initializes the instance variables. As a result the object
doesnot retain Javas implicit default values 0 and null. Then when the method display() is called it accesses the initialized instance variable and prints it.

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java
101 Asha


Process finished with exit code 0
```

Experiment 3: Parameterized Constructor

Program:

```
class Student3{
   int id;
   String name;

   Student3(int i,String n){
      id = i;
      name = n;
   }

   void display(){
      System.out.println(id+" "+name);
   }
}

public class Test3{
   public static void main(String[] args){
      Student3 s1 = new Student3(102,"Ravi");
      Student3 s2 = new Student3(103,"Meera");
      s1.display();
      s2.display();
   }
}
```

Explanation:

LOGIC: similar to the previous programs we have a class Student3 containing instance variables id and name.
We have a parameterized constructor Student3(int i, String n) which initializes the instance variables.
We have defined a method display() which accesses and displays the initialized values of id and name.
We have a public class Test3 with main functions where objects are created and with the creation of objects triggering the constructor the values are accordingly initialized.
The values for i and name are passed while creating an object as arguments that are received by the constructor parameters i and n.
the values are later displayed by calling the display function which accesses the initialized values.

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java
102 Ravi
103 Meera


Process finished with exit code 0
```

Experiment 4: Constructor Overloading

Program:

```
class Book{
   int id;
   String title;
   Book(){
      id=0;
      title="Not Assigned";
   }
   Book(int i, String t) {
      id = i;
      title = t;
   }
   void display(){
      System.out.println(id+" "+title);
   }
}
public class Test4{
   public static void main(String[] test4){
      Book b1 = new Book();
      Book b2 = new Book(201,"Java Basics");
      b1.display();
      b2.display();
   }
}
```

Explanation:

Constructor overloading is a concept in Java where more than one constructor is defined in the same class with different parameter lists.
This allows objects of a class to be initialized in different ways, depending on the arguments passed during object creation.
In this program, the class Book demonstrates constructor overloading by defining two constructors:
A default constructor with no parameters, which initializes id to 0 and title to "Not Assigned".
A parameterized constructor that takes an integer and a string as parameters and initializes id and title with the given values.
When the object b1 is created without passing any arguments, the default constructor is invoked.
When the object b2 is created by passing arguments, the parameterized constructor is invoked.
Thus, the same class uses different constructors to initialize objects in multiple ways, which is known as constructor overloading.

Here we create a class Book with two class constructors, the first constructor assigns values 0 for id and Not Assigned for title when object is created
The second constructor takes int i and String t as parameters and assigns it to id and title, Then the display method displays the id and title
In the main function Object b1 is created with values 0 and Not Assigned because no argument is passed on object creation
The object b2 is created with values 201 and Java Basics, both the objects values are printed by calling display method.

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/
0 Not Assigned
201 Java Basics

Process finished with exit code 0
```

Experiment 5: Constructor vs Method

Program:

```
class Employee{
   int id;

   Employee(int i){
      id=i;
      System.out.println("Constructor called ! ");
   }
   void setId(int i){
      id=i;
      System.out.println("Method called");
   }
}

public class Test5{
   public static void main(String[] test5){
      Employee e1 = new Employee(10);
      e1.setId(20);
   }
}
```

Explanation:

This program is based on comparision of Constructor vs Methods. In the program a constructor Employee is created which takes i as parameter and assigns to id;
We also have a method setId that does the same thing as constructor;
The main difference between these two are the constructor is called on object creation but method needs to be called explicitly to perform operations inside the method.
Thats what the obove program show us.

This program demonstrates the difference between a constructor and a method in Java.
The class Employee contains a parameterized constructor that takes an integer value and initializes the instance variable id. This constructor is automatically invoked when an object of the class is created.
The class also contains a method setId(), which performs a similar task of assigning a value to id, but it must be called explicitly using the object reference.
In the main method, when the object e1 is created, the constructor is called automatically and initializes id with the value 10.
Later, the method setId() is explicitly called to change the value of id to 20.
This program shows that constructors are used for object initialization at the time of creation, whereas methods are used to perform operations after the object has been created.

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/
Constructor called !
Method called

Process finished with exit code 0
```

Experiment 6: Multiple Objects – Constructor Called Each Time

Program:

```
class Counter{
    Counter(){
        System.out.println("Object created");
    }
}

public class Test6{
    public static void main(String[] test6){
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter c3 = new Counter();
    }
}
```

Explanation:

This program demonstrates that a constructor is called each time an object of a class is created.
The class Counter contains a default constructor that prints a message when an object is created.
In the main method, three objects (c1, c2, and c3) are created, so the constructor is invoked three times.
This proves that constructors are executed automatically during object creation.

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/
Object created
Object created
Object created

Process finished with exit code 0
```

Experiment 7: Constructor Ensuring Valid Object State

Program:

```
class Product{
   int price;

   Product(int p){
      if(p>0)
         price=p;
      else
         price=1;
   }
   void display(){
      System.out.println("Price= "+price);
   }
}
public class Test7 {
   public static void main(String[] test7){
      Product p1 = new Product(100);
      Product p2 = new Product(200);
      p1.display();
      p2.display();
   }
}
```

Explanation:

This program demonstrates the use of a constructor for data validation.
The class Product contains a parameterized constructor that checks whether the input value for price is valid.
If the value passed is greater than zero, it is assigned to price; otherwise, a default value of 1 is assigned.
This ensures that every object of the class is initialized with a valid price at the time of object creation.
The display() method is used to print the price of the product.

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/
Price= 100
Price= 200


Process finished with exit code 0
```

Experiment 8: Constructor Chaining (Using this)

Program:

```
class Demo{
   Demo(){
      this(10);
      System.out.println("Default constructor");
   }
   Demo(int x){
      System.out.println("Parameterized Constructor: "+x);
   }
}
public class Test8{
   public static void main(String[] test8){
      Demo d = new Demo();
   }
}
```

Explanation:

This program demonstrates constructor chaining in Java using the this() keyword.
The class Demo contains two overloaded constructors, where the default constructor calls the parameterized constructor using this(10) as its first statement.
When an object is created, the default constructor is invoked first, but due to constructor chaining, the parameterized constructor executes before the remaining statements of the default constructor.
As a result, the parameterized constructor prints its message first, followed by the default constructor.
This confirms that constructor chaining allows one constructor to reuse another constructor's logic, enforces a fixed execution order, and ensures proper object initialization.

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/
Parameterized Constructor: 10
Default constructor


Process finished with exit code 0
```

Experiment 9: Constructor Access Control

Program:

```
class Secure{
    private Secure(){
        System.out.println("Private Constructor");
    }
    static Secure createObject(){
        return new Secure();
    }
}
public class Test9{
    public static void main(String[] test9){
        Secure s = Secure.createObject();
    }
}
```

Explanation:

This program demonstrates the use of a private constructor in Java to restrict direct object creation.
The class Secure defines a private constructor, which prevents objects from being created using the new keyword from outside the class.
Instead, a public static method createObject() is provided, which internally calls the private constructor and returns a new object of the class.
In the main method, the object is created by invoking this static factory method rather than calling the constructor directly.
This approach ensures controlled object creation, improves security, and is commonly used in patterns such as factory methods and singleton design.

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/

Private Constructor


Process finished with exit code 0
```

Final Task

Program:

```
class LibraryBook{
   int bookId;
   String title;
   Boolean availability;
   private LibraryBook(){}

   LibraryBook(int i,String n){
      bookId=i;
      title=n;
      availability=false;
   }
   LibraryBook(int i,String n,Boolean check){
      bookId=i;
      title=n;
      availability=check;
   }
   void display(){
      System.out.println("Book Id: "+bookId+", Book Title: "+title+", Availability: "+availability);
   }
}
public class FinalTask{
   public static void main(String[] finaltask){
      LibraryBook b1 = new LibraryBook(100,"Stephen Hawkins");
      LibraryBook b2 = new LibraryBook(101,"Java Basics for DSA",true);
      b1.display();
      b2.display();
   }
}
```

Explanation:

The class LibraryBook represents a book with mandatory and optional attributes.
It contains three instance variables: bookId, title, and availability.
A private default constructor is defined to prevent the creation of a LibraryBook object without mandatory information such as bookId and title.
This ensures that objects cannot be instantiated using new LibraryBook() from outside the class.
Parameterized constructors are provided to enforce initialization of the mandatory fields at the time of object creation, while allowing flexibility for optional fields like availability.
As a result, every LibraryBook object is created in a valid and consistent state, ensuring data integrity and controlled object creation.

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/s
Book Id: 100, Book Title: Stephen Hawkins, Availability: false
Book Id: 101, Book Title: Java Basics for DSA, Availability: true

Process finished with exit code 0
```

ensuring data integrity and controlled object creation.

Viva in your own words:-
Basic level

1. What is a constructor?
⇒ A constructor is a special method that has same name as the class, is automatically called when an object is created and is used to initialize data members of a class.

2. Why do we use constructors?
⇒ Constructors are used to assign initial values to an object and ensure it is created in a valid state.

- What is a default constructor?

⇒ A default constructor is a constructor that takes no parameters and is called when an object is created without passing arguments.

- What is constructor chaining?

⇒ Constructor chaining is when one constructor calls another constructor using this() or super().

2. Why does a constructor not have return type?

⇒ A constructor does not have return type because its purpose is to initialize objects not to produce data.

3. When is a constructor executed?

⇒ A constructor is executed when object is created. It automatically executes on object creation.

Intermediate Level

4. What problems occur if constructors are not used?

⇒ Without constructors, object may be created without proper initialization, leading to invalid or inconsistent data.

5. How is constructor overloading different from method overloading?

⇒ Constructor overloading is used to initialize objects in different ways, while method overloading is used to perform different operations using same method name.

6. Why is constructor execution automatic but method execution manual?

⇒ Constructor execution is automatic because object initialization must happen before the object is used; whereas methods performs actions and are called only when needed.

Advanced / Conceptual level

7. How do constructors help maintain object Consistency?

⇒ Constructors ensure that all required data members are initialized with valid values when the object is created.

8. Why would you make a constructor private?

⇒ Constructor is made private to restrict or control object creation from outside the class.

9. Explain constructor chaining with an example.

⇒ Constructor chaining occurs when one constructor calls another constructor using this() or super().

Example:- A default constructor calling a parameterized constructor using this(10). (like in Test8 code)

10. Can a constructor call another constructor? How?

⇒ Yes a constructor can call another constructor using this() for the same class or super() for the parent class, and it must be the first statement.